

-Problem Description

This algorithm is used to find the **maximum possible product** of any three integers within a given array, accounting for both large positive integers and pairs of large negative integers that result in a positive product.

-Input and Output

- **Input:** An array of integers of size $n \geq 3$.
- **Output:** The maximum product as a single integer.

-Input Size

Let n be the number of elements in the input array.

- Basic Operation

The basic operation in this algorithm is **comparison** ($\text{num} > \text{max}$ and $\text{num} < \text{min}$) used to update the state variables.

-Does the Number of Operations Vary?

- No.
- The algorithm performs a fixed set of comparisons for every element in the array regardless of its value. Therefore, the Best, Average, and Worst cases are asymptotically identical.

-Recurrence Relation

Let $T(n)$ denote the running time of the algorithm for an input size n .

Base Case

$$T(0) = \Theta(1)$$

(Reached when $\text{index} == \text{length of array}$) .

Recursive Case

$$T(n) = T(n-1) + c$$

Where c represents the constant time taken to perform comparisons and updates for a single element.

-Solving the Recurrence

Method Used: Iteration Method.

Expansion step-by-step:

$$\begin{aligned}T(n) &= T(n-1) + c \\&= [T(n-2) + c] + c = T(n-2) + 2c \\&= [T(n-3) + c] + 2c = T(n-3) + 3c \\&= T(n-4) + 4c \\&\vdots \\T(n) &= T(n-k) + kc\end{aligned}$$

Determining the **base case**:

The condition for the base case **$T(0)$** is met when:

$$n - k = 0 \Rightarrow k = n$$

Substituting $k = n$

$$T(n) = T(n-n) + nc = T(0) + nc = \theta(1) + cn$$

$$T(n) = \theta(n)$$

- Time Complexity

Based on the derivation above, the time complexity of the algorithm is: $\Theta(n)$

- Space Complexity

The space complexity is determined by the **recursion stack**.

- **Recursion Depth:** The algorithm calls itself once for every element in the array, resulting in a depth of n .
- **Space per Frame:** Each frame stores a constant number of variables (max 1..3, min 1..2).

Therefore,

Space Complexity = $\Theta(n)$